

Problema del productor-consumidor con prioridades,
caducidades y tiempos fijos

Nicolás Corbal Capas

Hugo Veiga Couselo

Introducción

La concurrencia es una característica fundamental en los sistemas operativos modernos y un factor clave para mejorar el rendimiento. Sin embargo, el acceso concurrente a recursos compartidos hace necesario el uso de mecanismos de sincronización. En muchos casos se establece una jerarquía de prioridades de modo que aquellos elementos más prioritarios deben ser procesados preferentemente, y adicionalmente los datos pueden tener una vigencia limitada, como sucede con los paquetes de red con un TTL que deben descartarse si permanecen demasiado tiempo en la cola.

A diferencia de los casos anteriores, en este escenario los productores no generan elementos a una velocidad aleatoria, sino que lo hacen a una cadencia fija determinada por su prioridad. De forma análoga, el consumidor tarda un tiempo fijo en procesar cada ítem en función de la prioridad a la que pertenezca. Esta restricción temporal permite estudiar de manera más controlada la interacción entre la jerarquía de prioridades y la caducidad de los elementos, y evaluar si la solución mantiene su corrección y estabilidad bajo un régimen determinista.

En el presente informe se documenta la implementación de una solución a este problema utilizando mutexes y variables de condición en lenguaje C, reutilizando las estructuras desarrolladas para los opcionales anteriores y añadiendo una gestión de tiempos fijos por prioridad mediante llamadas a `sleep()`. Se analiza el funcionamiento de la solución, las decisiones de diseño adoptadas y las conclusiones obtenidas tras su ejecución.

Forma de ejecución

Primeramente, para compilar el código un ejemplo de comando de compilación usando el compilador gcc es:

```
gcc -Wall -I ./ prod_cons_tiempos.c buffer.c -o ejecutable -lpthread
```

Siendo el ejecutable ejecutado simplemente con:

./ejecutable

Funcionamiento del código

La responsabilidad del código vuelve a estar dividida entre 3 archivos, `buffer.c`, `buffer.h` y `prod_cons_tiempos.c`. El archivo `buffer.c` contiene las funciones necesarias para la producción y la consumición, el archivo de cabecera el tipo de dato usado como forma de acceso común y `prod_cons_tiempos.c` la responsabilidad de la solución del problema.

En el tipo de dato `datos` se recoge lo necesario para la cola (un buffer, marcadores de inicio y fin y el contador cantidad) y se añaden, como en los opcionales anteriores, el mutex y las variables de condición `condc` y `condp` para evitar el uso de globales y facilitar un diseño con un buffer por prioridad. El campo `prioridade` identifica el buffer al que está asociado cada productor. El tipo de dato `item` es una estructura que almacena el valor entero, el instante de creación en milisegundos y el tiempo de caducidad en segundos, de modo que el consumidor pueda decidir si el elemento sigue siendo válido.

La función auxiliar `tempo_ms()` declarada en `buffer.c` obtiene el instante actual mediante `clock_gettime` con `CLOCK_MONOTONIC`, lo que proporciona una referencia temporal estable e independiente de cambios en la hora del sistema. La función `produce_item()` lee un número del archivo `numeros<prioridad>.txt`, asigna la marca de tiempo de creación y genera un valor de caducidad aleatorio entre 1 y 12 segundos, devolviendo el resultado a través de un parámetro de salida.

El archivo `prod_cons_tiempos.c` se divide en funciones. La función `main` se encarga de inicializar los tres buffers, sus mutexes y variables de condición, lanzar los tres hilos productores y el consumidor, esperar a su finalización y liberar los recursos. El hilo productor, a diferencia de los casos anteriores, no simula su velocidad con un `sleep` aleatorio, sino con un `sleep` fijo igual al valor de su prioridad: P1 produce cada 1 s, P2 cada 2 s y P3 cada 3 s. Tras esperar, produce un item, adquiere el mutex del buffer asociado, se bloquea en `condp` si el buffer está lleno, inserta el elemento, señala a `condc` y libera el mutex. Al finalizar las iteraciones inserta un item centinela con valor 0 para notificar el fin al consumidor.

El hilo consumidor mantiene la estrategia de los opcionales anteriores: recorre los buffers por orden de prioridad e intenta adquirir cada mutex con `pthread_mutex_trylock()`, lo que le permite saltar a un buffer de prioridad inferior sin bloquearse cuando el actual está ocupado o vacío. Al consumir un item comprueba si `tempo_ms() - t_creacion` supera la caducidad expresada en milisegundos y, en caso afirmativo, lo descarta y lo contabiliza

en el vector caducados. Si el ítem es válido, lo suma al total de su prioridad y simula el procesamiento con un sleep fijo de 3-p segundos, de modo que los elementos de P1 tardan 3 s en procesarse, los de P2 2 s y los de P3 1 s, generando así un escenario en el que el productor rápido es consumido lentamente y viceversa.

La variable `feito` se emplea para reiniciar la exploración cada vez que se consume un elemento, garantizando que tras procesar un ítem de prioridad n se vuelva a comprobar primero el buffer de prioridad 1, respetando así la política jerárquica. Si tras recorrer los tres buffers no se ha consumido nada, se cede la CPU con un `usleep` breve para evitar la espera activa. La finalización se gestiona mediante el vector `fin`, que se marca al recibir el centinela de cada productor.

Los resultados observados muestran que el productor P1, pese a generar elementos cada segundo, se ve limitado por el tiempo de consumo de 3 s, lo que provoca que el buffer se llene con rapidez y se produzcan bloqueos frecuentes en `condp`. En cambio, P3 produce cada 3 s y se consume en 1 s, de manera que su buffer permanece casi siempre vacío. Esta asimetría favorece la aparición de caducidades en los buffers más prioritarios, lo que resulta ilustrativo del compromiso entre prioridad y plazo.

En la siguiente captura se recoge una ejecución completa de la solución, en la que pueden observarse las inserciones de los tres productores con su correspondiente caducidad, los consumos efectivos, los descartes por expiración y las sumas finales por prioridad.

prod_cons_tiempos - Terminal

```
$ gcc -Wall -I./ prod_cons_tiempos.c buffer.c -o prod_cons -lpthread  
$ ./prod_cons
```

```
[2081819943] P1 insire 10 (caduc 9s)  
[2081819947] Cons consume 10 de P1 (3s)  
[2081820946] P2 insire 11 (caduc 2s)  
[2081820948] P1 insire 20 (caduc 12s)  
[2081821941] P3 insire 13 (caduc 1s)  
[2081821948] P1 insire 30 (caduc 7s)  
[2081822949] Cons consume 20 de P1 (3s)  
[2081822951] P2 insire 22 (caduc 10s)  
[2081822953] P1 insire 15 (caduc 2s)  
[2081823958] P1 insire 25 (caduc 10s)  
[2081824943] P3 insire 26 (caduc 2s)  
[2081824956] P2 insire 33 (caduc 2s)  
[2081824961] P1 insire 35 (caduc 8s)  
[2081825954] Cons consume 30 de P1 (3s)  
[2081825966] P1 insire 12 (caduc 10s)  
[2081826961] P2 insire 44 (caduc 8s)  
[2081826971] P1 insire 18 (caduc 4s)  
[2081827947] P3 insire 39 (caduc 5s)  
[2081827976] P1 FIN suma = 165  
[2081828956] Cons DESCARTADO 15 de P1 (caduc 2s)  
[2081828956] Cons consume 25 de P1 (3s)  
[2081828965] P2 insire 55 (caduc 5s)  
[2081830951] P3 insire 52 (caduc 5s)  
[2081830968] P2 insire 66 (caduc 8s)  
[2081831959] Cons consume 35 de P1 (3s)  
[2081832973] P2 insire 77 (caduc 6s)  
[2081833956] P3 insire 65 (caduc 8s)  
[2081834964] Cons consume 12 de P1 (3s)  
[2081834978] P2 FIN suma = 308  
[2081836961] P3 insire 78 (caduc 5s)  
[2081837969] Cons DESCARTADO 18 de P1 (caduc 4s)  
[2081837969] Cons fin de P1  
[2081837969] Cons DESCARTADO 11 de P2 (caduc 2s)  
[2081837969] Cons DESCARTADO 22 de P2 (caduc 10s)  
[2081837969] Cons DESCARTADO 33 de P2 (caduc 2s)  
[2081837969] Cons DESCARTADO 44 de P2 (caduc 8s)  
[2081837969] Cons DESCARTADO 55 de P2 (caduc 5s)  
[2081837969] Cons consume 66 de P2 (2s)  
[2081839965] P3 insire 14 (caduc 2s)  
[2081839972] Cons DESCARTADO 77 de P2 (caduc 6s)  
[2081839972] Cons fin de P2  
[2081839972] Cons DESCARTADO 13 de P3 (caduc 1s)  
[2081839972] Cons DESCARTADO 26 de P3 (caduc 2s)  
[2081839972] Cons DESCARTADO 39 de P3 (caduc 5s)
```

Conclusiones

La solución implementada asegura la ausencia de carreras críticas y respeta la política de prioridades, al tiempo que incorpora un esquema de tiempos fijos que permite estudiar de forma más controlada el comportamiento del sistema. La combinación de mutexes, variables de condición y `pthread_mutex_trylock()` en el consumidor permite atender preferentemente los elementos de mayor prioridad sin incurrir en bloqueos innecesarios ni en espera activa.

La introducción simultánea de caducidades y tiempos fijos pone de manifiesto una tensión inherente al sistema: cuando el tiempo de consumo supera al de producción en las prioridades altas, los buffers se saturan y aumenta el número de elementos descartados por expiración. En escenarios reales, como el procesado de paquetes de red con TTL o la gestión de datos de sensores con vigencia limitada, este efecto puede suponer una pérdida significativa de información y obliga a dimensionar adecuadamente los recursos del consumidor.

En conjunto, la solución implementada es correcta, robusta y extensible. Constituye una evolución natural de los opcionales anteriores, añadiendo determinismo temporal a la gestión de prioridades y caducidades y evidenciando que el correcto funcionamiento de un sistema concurrente no depende únicamente de la sincronización, sino también del equilibrio entre la velocidad de producción y la capacidad de consumo.